



COGNITIVE INDUSTRIES
LES INDUSTRIES COGNITIVES

Architecture technique d'AegisTrace

Implémentation de référence de la norme AI-IDP

Pierre-Edward Procyk
Fondateur / PDG
Cognitive Industries — Les Industries Cognitives
Saguenay, Québec, Canada
p.procyk.media@gmail.com

Document ID: AI-IDP-ARCHFR-2.0.0
Version: 2.0.0
Date: 2026-08-01
Classification: Public

1. Aperçu

AegisTrace est l'implémentation de référence de la norme AI-IDP. Il démontre l'identité permanente des acteurs d'IA, la délégation, la provenance, la traçabilité, les preuves de qualité, la responsabilité et l'audit permanent de bout en bout. L'architecture est neutre à l'égard des fournisseurs, sensible à la juridiction (canadienne d'abord) et construite sur des primitives cryptographiques établies.

AegisTrace est implémenté en Python 3.12 avec typage statique strict, signatures Ed25519 via la bibliothèque cryptography, validation JSON Schema via jsonschema, serveur HTTP FastAPI et stockage SQLite local (avec une conception de production compatible PostgreSQL). L'implémentation est testable, reproductible et déployable dans des environnements conteneurisés.

2. Architecture en couches

L'architecture s'organise en six couches, chacune ayant une responsabilité claire et des interfaces bien définies avec les couches adjacentes.

2.1 Couche d'identité

La couche d'identité gère les identifiants d'acteurs d'IA et les clés de signature. Elle comprend trois modules :

- `identity/ids.py` — la classe `Identifier`, l'analyse et la génération d'URI, les identifiants d'événement basés sur ULID, et la génération de slug. Les identifiants suivent le format `aitrace://<juridiction>/<type-entité>/<slug>[#<version-ou-instance>]` et sont permanents et uniques.
- `identity/keys.py` — les classes `KeyService` et `KeyRecord`. Les clés Ed25519 sont créées, rotationnées, suspendues, révoquées et terminées. Les identifiants de clé sont permanents et uniques. Les clés révoquées ou terminées ne peuvent pas produire de nouveaux événements valides ; les clés rotationnées demeurent vérifiables.
- `identity/lifecycle.py` — les classes `Registry` et `EntityRecord`. Les entités transitent par des états (proposé, actif, suspendu, révoqué, terminé, archivé), toutes les transitions étant enregistrées. Les identifiants permanents demeurent résolubles après la terminaison, la révocation, la fermeture du fournisseur, la retraite du modèle, le transfert de dépôt et la restructuration organisationnelle.

2.2 Couche de registre

La couche de registre est le cœur du système de traçabilité. Elle comprend :

- `ledger/append_only.py` — la classe `AppendOnlyLedger`. Les événements sont à ajout unique ; la modification et la suppression sont interdites. Le `previous_event_hash` de chaque événement correspond au `event_hash` de l'événement précédent, formant une chaîne de hachage. Le registre vérifie l'intégrité de la chaîne, l'exactitude du hachage d'événement et la validité de la signature.
- `ledger/merkle.py` — calcul de racine de Merkle et de preuve de Merkle sur les séquences d'événements. Les racines de Merkle périodiques sont ancrées à des surfaces de vérification publiques.
- `ledger/batched.py` — registre à traitement par lots avec journal de write-ahead (WAL) pour un débit plus élevé en production.
- `signing/ed25519.py` — opérations de signature Ed25519 et hachage SHA-256 / BLAKE2b.
- `signing/canonical.py` — canonisation JSON déterministe pour la stabilité de la signature.
- `signing/pqc.py` — interface de migration de signature post-quantique (ML-DSA-65, SLH-DSA-128s) selon FIPS 204 et FIPS 205.

2.3 Couche de gouvernance

La couche de gouvernance gère les chaînes d'autorité. Elle comprend :

- `delegation/broker.py` — la classe `DelegationBroker`. Les agents parents délèguent des portées d'autorité bornées aux agents enfants. Les délégations sont révocables et expirables. Chaque agent enfant se résout en une délégation parent ; la chaîne de délégation est vérifiable de bout en bout.
- `authorization/engine.py` — les classes `PolicyEngine`, `Authorization` et `Approval`. Les autorisations enregistrent qu'un principal a autorisé une action ; les approbations sont des enregistrements à usage unique pour les actions à fort impact. Le moteur de politiques évalue les demandes d'autorisation par rapport à la politique courante, applique la portée et refuse les actions à échec fermé lorsque les vérifications sont inconclues.

2.4 Couche d'événement

La couche d'événement construit, signe et enregistre les événements. Elle comprend :

- `events/models.py` — les classes de données `Event`, `Actor` et `ExecutionContext`. Le vocabulaire d'action (75 actions canoniques de DISCOVER à TRIGGER_EXTERNAL_EFFECT), les niveaux de visibilité (PUBLIC, CONTROLLED, ORGANIZATION_PRIVATE, SEALED) et la version de schéma sont définis ici.
- `events/collector.py` — la classe `EventCollector`. Construit les événements, les signe avec des clés actives liées à l'agent ou au contrôleur de l'acteur, calcule les hachages d'événement, les ajoute au registre et renvoie l'objet Event typé.

2.5 Couche d'adaptateur

La couche d'adaptateur fait le pont entre AegisTrace et les systèmes externes. Elle comprend :

- `adapters/filesystem.py` — l'adaptateur de système de fichiers. Enregistre les opérations de système de fichiers (CREATE, MODIFY, DELETE) avec des hachages SHA-256 avant/après. Fournit une protection contre le parcours de chemin.
- `adapters/git.py` — l'adaptateur Git. Encapsule les opérations Git (commit, branch, merge, tag) et produit des références de ressources compatibles AegisTrace.

- `adapters/github.py` — l'adaptateur de preuve GitHub. Produit les charges utiles pour le modèle de preuve à double dépôt : attestations de commit, ancres de racine Merkle, preuves Merkle, statut de révocation, attestations de version.
- `adapters/github_remote.py` — intégration GitHub distante en direct. Pousse les ancres Merkle, le statut de révocation et le statut de certification vers un dépôt de vérification public via l'API GitHub REST. Pousse les preuves chiffrées vers un dépôt de preuves privé. Authentification via la variable d'environnement `AEGISTRACE_GITHUB_TOKEN`.
- `adapters/database.py` — l'adaptateur de base de données. Encapsule les opérations SQLite (`WRITE`, `DELETE`, `ALTER_SCHEMA`) avec des hachages avant/après.
- `adapters/mcp.py` — l'adaptateur MCP. Enregistre les invocations d'outils MCP.
- `adapters/otel.py` — exportateur OpenTelemetry. Exporte les événements AegisTrace comme spans OTel pour l'observabilité opérationnelle.

2.6 Couche de stockage

La couche de stockage persiste le registre, le registre d'entités et les clés. Elle comprend :

- `storage/sqlite.py` — la classe `SQLiteStorage`. Fournit un accès indexé aux événements, aux enregistrements de registre et aux clés. Le registre est également persisté en JSONL pour la lisibilité humaine et la vérification externe.
- `storage/postgres.py` — la classe `PostgresStorage`. Backend PostgreSQL pour les déploiements de production multi-locataires. Utilise des colonnes JSONB pour l'évolution flexible du schéma, BIGSERIAL pour l'ordonnancement des séquences, et des index sur les chemins de requête les plus courants. SSL requis par défaut.
- Fichiers JSONL — la forme canonique, lisible par l'humain et vérifiable de l'extérieur du registre.

3. Propriétés de sécurité

AegisTrace applique les propriétés de sécurité suivantes, toutes vérifiées par des tests automatisés dans `tests/security/` : résistance à la falsification ; application de la révocation ; intégrité de la chaîne de hachage ; exactitude du hachage d'événement ; vérification de signature ; approbation à usage unique ; application de la portée ; fonctionnement à échec fermé ; protection contre le parcours de chemin ; séparation public/privé.

4. Propriétés de confidentialité

AegisTrace applique les propriétés de confidentialité suivantes, toutes vérifiées par des tests automatisés dans `tests/privacy/` : identifiants pseudonymes ; aucune donnée de contact dans les événements ; niveaux de visibilité (`PUBLIC`, `CONTROLLED`, `ORGANIZATION_PRIVATE`, `SEALED`) ; enregistrements scellés ; prévention de l'énumération d'utilisateurs.

5. Propriétés de permanence

AegisTrace applique les propriétés de permanence suivantes, toutes vérifiées par des tests automatisés dans `tests/permanence/` : résolubilité permanente ; validité historique des signatures ; préservation de la rotation des clés ; préservation de l'identité à travers les changements de modèle et de fournisseur ; préservation de l'archivage ; permanence du registre après la terminaison de l'agent.

6. Propriétés de conformité

AegisTrace applique les propriétés de conformité suivantes, toutes vérifiées par des tests automatisés dans `tests/conformance/` : conformité au schéma ; vocabulaire d'action canonique ; niveaux de visibilité canoniques ; invariants requis ; ajout unique ; séparation des champs de niveau public ; intégrité du registre après de nombreux ajouts.

7. Outils en ligne de commande

AegisTrace fournit quatre outils en ligne de commande : `verify` (vérifie l'intégrité du registre) ; `audit` (produit un sommaire d'audit) ; `reconstruct` (reconstitue une chronologie d'incident) ; `admin demo` (exécute un scénario de démonstration complet).

8. Niveaux de conformité

AI-IDP définit quatre niveaux de conformité (L1 à L4) qui sont cumulatifs. L1 (référence) exige des identifiants permanents, des événements signés, un registre local. L2 (standard) ajoute la vérification publique, les preuves privées, la délégation, l'autorisation, l'approbation. L3 (haute assurance) ajoute la réplication d'archivage indépendante, la fédération, les tests de confidentialité et de permanence, l'audit de conformité annuel. L4 (assurance maximale) ajoute la double approbation, le coffre-fort contrôlé par le régulateur, l'ancrage de journal de transparence en temps réel, la préparation post-quantique, les audits trimestriels de sécurité.

9. Migration post-quantique

L'interface de signature est abstraite. Ed25519 peut être migré vers un schéma post-quantique (ML-DSA-65 selon FIPS 204, ou SLH-DSA-128s selon FIPS 205) lorsque les bibliothèques standardisées seront disponibles. La migration est documentée dans `spec/PERMANENT_RECORD_PROTOCOL.md` : un nouveau schéma de signature est adopté, les signatures existantes sont resignées avec le nouveau schéma, la resignation est enregistrée comme un nouvel événement signé, et les signatures originales sont préservées. La v2.0.0 inclut les classes `MLDSA65Scheme` et `SLHDSA128sScheme` complètes d'interface dans `src/aegistrace/signing/pqc.py`.

10. Conclusion

L'architecture technique d'AegisTrace fournit une implémentation de référence complète, fonctionnelle et testée de la norme AI-IDP. Elle applique tous les invariants requis, prend en charge toute la couverture d'action requise, prend en charge les quatre niveaux de visibilité du registre, prend en charge le fonctionnement hors ligne avec réconciliation et prend en charge la fédération. L'architecture est neutre à l'égard des fournisseurs, sensible à la juridiction et construite sur des primitives cryptographiques établies. Elle convient comme base pour une Norme nationale du Canada, un cadre de certification sectoriel ou un système de responsabilité d'entreprise.